

DevAstra — Master End-to-End Improvement Plan

Consolidates: *Architectural Audit Report + Functional Improvements Report (Judge's Perspective)*

Purpose: One sequenced execution plan — not a re-listing of findings, but the order to actually do the work in, with dependencies made explicit so fixes don't create new bugs (e.g., adding role-based dashboards before adding role-based access control would open a security hole).

Guiding logic for the sequencing below

Three rules drove the ordering, because doing these out of order actively causes new problems:

1. **Cleanup before building.** Deleting the dead Django backend and debug scripts costs ~20 minutes and touches nothing functional — do it first so the rest of the work happens in a clean, legible repo.
2. **Schema + security ship together, never schema-then-security-later.** The audit specifically flagged that adding real data behind the Industry/Institution dashboards *before* adding role-based route guards and RLS would turn a dormant privilege-escalation

bug into a live one. Every phase below that adds a table also adds its RLS/guard in the same phase.

3. **Fix broken before fix fake.** A crash or dead button (job application, institution dashboard) loses more judge trust than a hardcoded number does. Broken-action bugs are fixed before mock-data-replacement work.

Phase 0 — Repo Cleanup (≈30 min, do this first, zero risk)

No functional risk, pure credibility/legibility gain. From: Architectural Audit §1.1, §8.

- [] Delete or archive `backend/` (dead Django project), `requirements.txt`, `Procfile`, `runtime.txt`.
- [] Remove `db.sqlite3` from the repo and add it to `.gitignore`.
- [] Move the ~35 root-level debug scripts (`test-oauth*.js`, `test-login*.js`, `puppeteer-*.js`, etc.) into `/archive` or delete.
- [] Move the ~14 patch scripts (`fix_*.py`, `update_*.py`, `apply_edits.py`, `edit_css.py`, `replace_theme.py`) into `/archive` once confirmed their edits are already reflected in the current source (they are — they already ran once).
- [] Consolidate the 3 near-duplicate MCQ JSON files and the ~5 seed script generations into one canonical

seed pipeline; run it once against the live DB and verify question counts match what you expect.

- [] Fix the broken `` in `README.md` and remove duplicate logo files.
- [] Sweep stray `console.log`s that print full profile payloads (`AuthContext.completeOnboarding`, `auth.service.updateProfile`).

Phase 1 – Stop the Bleeding: Critical Broken-Action Fixes (highest combined priority in both reports)

These are the items both reports independently rank #1 – the audit calls them P0 "will break in front of a judge," the functional report calls them the top Tier-1 click-path failures.

- [] **Create the `job_applications` table** (`student_id`, `job_id`, `status`, `applied_at`, + FK indexes) and add RLS: students can insert/select their own rows; a future company-owner check gates SELECT for recruiters. Wire `jobService.apply()` and `jobService.getApplications()` to it for real.
- [] **Fix the Institution Dashboard's broken fallback logic.** One-line change:
`setData(Object.keys(res.data).length ? res.data : mockData)` instead of `res.data || mockData`. This alone moves the page from

"blank/error" to "renders your intended mock UI" while the real-data work in Phase 3 catches up.

- **[] Fix or guard `DailyPlanner.jsx` against `plannerData.targets` being undefined** when the stub returns `[]`. Either add a null-check/empty-state, or redirect `/planner` to the Dashboard's own planner widget (which already pulls real data from `assessments / user_assessments` and works).
- **[] Wire "Post Job" on the Industry Dashboard to a real insert against `jobs`** (the currently-commented-out call). Listing already reads real data — closing this one gap makes the whole Jobs feature end-to-end real.

Checkpoint: after Phase 1, a judge can complete the entire student-side job-seeking loop (browse → apply → see application recorded) and the Institution Dashboard no longer visibly breaks. This is the single highest-leverage checkpoint in the whole plan — if time runs out anywhere after this, you're still in reasonable shape.

Phase 2 — Role System: Schema + Security Together

Audit §3.1–3.3 and Functional Report Tier-1 #4 both point at the same root cause: there's no real way to become Industry/Institution, and the access-control gap around it is currently dormant only because nothing real sits behind it. Both halves ship in this phase, together, on purpose.

- [] Add a **role selector at signup** (Student / Industry / Institution — self-declared is fine for a hackathon demo; a verification/approval step is a stretch goal, not a blocker).
- [] Add `companies` and `institutions` profile tables, linked to `users` via the existing `role` field plus a new FK (`org_id`).
- [] **Lock down role in updateProfile** — remove it from the generically-writable `userColumns` list; require a separate, more controlled path (an RPC, or a one-time set-at-registration field) so a student can no longer self-promote by calling the existing profile-update endpoint.
- [] Add a **route-level RoleRoute guard component** for `/admin/institution` and `/admin/industry`, replacing the current pattern where the only gate is an in-component `if (!isIndustry)` conditional.
- [] Add matching RLS policies keyed to `role / org_id` for every new table introduced from here on — no table ships without its policy in the same commit.

Checkpoint: all three personas (Student, Industry, Institution) can now be created and demoed live, back-to-back, in one sitting — directly answering the judge question *"can I sign up as a company right now?"* with "yes, watch."

Phase 3 — Complete the Core Three-Role Loop + Named PS Features

This is where the Tier-1/Tier-2 gaps from the Functional Report and the "8 of 14 PS requirements missing" finding from the Audit get closed. Build in this order — each item is the minimum viable version, not the full spec.

- [] **Applications-received view for Industry** — even a raw list ("N students applied, view profile") against the `job_applications` table from Phase 1. This is the other half of the application loop and a named PS requirement ("application tracking and recruitment management").
- [] **Real Institution Analytics** — replace the Phase-1 patched mock with actual aggregate queries over `user_assessments` / `users` (average skill score, top skill gaps, career distribution). Reuses data that already exists; no new tables needed.
- [] **Digital portfolio (minimal)** — a `portfolio_items` table + a section on the Profile page listing completed assessments automatically, plus a manual "add project/certificate" field. Named explicitly in the PS, cheap relative to its visibility.
- [] **Distinguish internships from jobs** — either a `type` field on `jobs` or a separate lightweight table, so the platform can genuinely claim to serve "internships, apprenticeships, and entry-level jobs" as the PS specifies, not just generic listings.

- **[] Personalize the Roadmap** — replace the always-identical 3-node fallback with something derived from the user's actual `skills` /assessment history, even a simple rule-based mapping. This closes the Functional Report's Tier-3 #10 ("looks fake under scrutiny") risk.
- **[] Wire the Student Analytics page to real data** — swap the hardcoded chart arrays for actual `user_assessments` history and `job_applications` counts (from Phase 1). Closes Tier-3 #11.
- **[] Add lightweight placeholders for FDPs / mentorship / workshops / faculty internships** — not full features, just a "Coming Soon" section with a one-paragraph description of the intended flow. Costs under an hour; closes an otherwise-total silence on requirements an academic-institution judge is likely to ask about directly.
- **[] Un-stub or clearly gate** `learningService` / `notificationService` — either wire `LearningResources.jsx` to a real (even small, curated) `learning_resources` table, or change its empty state to explicitly say "coming soon" instead of silently showing nothing.

Checkpoint: after this phase, the PS requirement coverage table moves from "3 fully real / 3 partial / 8 missing" to something closer to "10+ fully or minimally real, 2–3 explicitly scoped as upcoming" — which is a materially different, much stronger story to tell a judge.

Phase 4 — Security Hardening Pass

Audit §2.3, §3.4–3.6. Lower demo-visibility than Phases 1–3, but do this before presenting to any technically-literate judge or panel, and definitely before any post-hackathon deployment.

- [] Verify RLS actually covers `questions` (not just `users / jobs / user_assessments`) so `correct_option` can't be read directly, bypassing the scoring RPCs.
- [] Move `pg` (raw Postgres driver) and any DB-credential-touching scripts out of `frontend/package.json` entirely — into a separate `/scripts` package that never ships in the Vite bundle.
- [] Re-run a Backend Advisor / RLS audit pass on every table added in Phases 1–3 (this project has already regressed the `SECURITY DEFINER search_path` fix once — treat this as a recurring checklist item, not a one-time fix).
- [] Confirm `.env.local` is actually git-ignored (verify, don't assume — `.gitignore` discipline is already known to be inconsistent given `db.sqlite3` was committed).

Phase 5 — AI Reliability & Demo Safety

Audit §5, Functional Report Tier-3 #12–13.

- [] Confirm the **deployed** edge function's Gemini model string matches what you intend to present (repo says `gemini-2.0-flash`, coordination notes say `gemini-3.6-flash` — check the live deployment directly, don't trust the repo).
 - [] Add a dev-only visible indicator (console flag or small badge) distinguishing live Gemini output from the hardcoded fallback text, so presenters always know which mode is active during the demo.
 - [] Replace `resume_tailor`'s fragile line-1-is-the-score string parsing with structured JSON output (matching the pattern already used correctly for interview scoring).
 - [] Add a simple per-user cooldown on AI actions to avoid burning through API quota during rapid judge testing.
 - [] As a team, agree on the honest answer to "is this chart/roadmap/score live or mocked" for anything not fully wired by Phase 3 — decide this in advance, not in the room.
-

Phase 6 — UX, Performance & Accessibility Polish

Audit §6, Functional Report closing note. Do this after functionality is solid — polish on top of broken features doesn't move the score.

- [] Cut the preloader from 8s back to ≤ 2 s and restore the skip button.
 - [] Make the global click-sound effect opt-in (Settings toggle), default off.
 - [] Test the full app on your actual (older/lower-spec) demo hardware, unplugged, with the usual background apps open — trim particle count / blur intensity if there's visible frame drop.
 - [] Add `prefers-reduced-motion` handling alongside the existing `prefers-color-scheme` handling.
 - [] Run a contrast check on the glass/blur dark-mode palette specifically — this effect is a common WCAG contrast failure pattern.
 - [] Verify the 768–900px responsive breakpoint range (recently migrated via scripted find-replace) doesn't have a dead zone in the nav.
 - [] Consolidate the CSS token layer (`colors.css` / `typography.css` / `spacing.css`) now that the one-off variable-rename scripts have served their purpose.
-

Phase 7 — Testing & Final Demo Dry-Run

Neither report found any automated test coverage. This phase exists because everything above is exactly the kind of change that silently regresses without it.

- [] Add a minimal smoke test (Playwright is already a dependency): login → dashboard → click every nav item → assert no error boundary fires. This single test would have caught the Institution Dashboard bug and the Daily Planner crash risk automatically — write it once, run it before every future demo.
- [] Add a basic test/manual-check for the RPC functions (`check_single_answer` , `submit_assessment_secure`) given this exact area has already regressed once via a `search_path` fix.
- [] Run the full judge walkthrough from the Functional Report yourself, end to end, on a clean/incognito session, on the actual demo machine, at least once before presenting.
- [] Rehearse the "show me the industry side" and "show me the institution side" asks specifically — these are the two questions most likely to expose whatever's left unfinished.

Consolidated Priority Matrix

For quick reference — every phase's items in one table, ranked by combined judge-visibility × security-risk.

Priority	Item	Phase	Source
● Critical	Job application flow (table + wiring)	1	Both

Priority	Item	Phase	Source
 Critical	Institution Dashboard crash fix	1	Both
 Critical	Daily Planner crash guard	1	Both
 Critical	Industry "Post Job" real insert	1	Both
 High	Role selector at signup	2	Functional
 High	role privilege-escalation lockdown	2	Audit
 High	RoleRoute guards on admin pages	2	Audit
 High	Applications-received view (Industry)	3	Functional
 High	Real Institution Analytics	3	Both
 Medium	Digital portfolio	3	Both
 Medium	Internship vs. job distinction	3	Both
 Medium	Roadmap personalization	3	Functional

Priority	Item	Phase	Source
 Medium	Student Analytics real data	3	Functional
 Medium	FDP/mentorship placeholders	3	Both
 Lower	RLS coverage on questions	4	Audit
 Lower	pg dependency relocation	4	Audit
 Lower	AI live/fallback indicator	5	Both
 Lower	Preloader/animation trim	6	Both
 Housekeeping	Delete dead backend, scripts, sqlite	0	Audit
 Housekeeping	Smoke test + demo dry-run	7	Both

If time is very short: minimum viable sequence

If only a few hours remain before the deadline, do **Phase 0**
→ **Phase 1** → **the first two bullets of Phase 2 (role selector + RoleRoute guard, skip the deeper RLS/ org_id work)** → **the first two bullets of Phase 3 (applications-received view + real Institution Analytics)**, then go straight to Phase 7's dry-run. That combination fixes every actively-broken action, makes all three personas demoable, and closes the two biggest "this is just mock data" exposures — the highest score-per-hour path available.

This plan assumes the two source reports (Architectural Audit, Functional Improvements) as ground truth for findings; this document only sequences and consolidates them into execution order.